Chinese Journal of Aeronautics, (2024), 37(6): 293–306



Chinese Society of Aeronautics and Astronautics
& Beihang University

Chinese Journal of Aeronautics

cja@buaa.edu.cn
www.sciencedirect.com



FULL LENGTH ARTICLE

MADRL-based UAV swarm non-cooperative game under incomplete information



Ershen WANG [a], **Fan LIU** [a], **Chen HONG** [b,*], **Jing GUO** [a], **Lin ZHAO** [c], **Jian XUE** [c], **Ning HE** [d]

[a] *School of Electronic and Information Engineering, Shenyang Aerospace University, Shenyang 110136, China*
[b] *College of Robotics, Beijing Union University, Beijing 100101, China*
[c] *School of Engineering Science, University of Chinese Academy of Sciences, Beijing 100049, China*
[d] *College of Smart City, Beijing Union University, Beijing 100101, China*

Received 19 June 2023; revised 3 August 2023; accepted 8 September 2023
Available online 25 March 2024

KEYWORDS

UAV swarm;
Reinforcement learning;
Deep learning;
Multi-agent;
Non-cooperative game;
Nash equilibrium

Abstract Unmanned Aerial Vehicles (UAVs) play increasing important role in modern battlefield. In this paper, considering the incomplete observation information of individual UAV in complex combat environment, we put forward an UAV swarm non-cooperative game model based on Multi-Agent Deep Reinforcement Learning (MADRL), where the state space and action space are constructed to adapt the real features of UAV swarm air-to-air combat. The multi-agent particle environment is employed to generate an UAV combat scene with continuous observation space. Some recently popular MADRL methods are compared extensively in the UAV swarm non-cooperative game model, the results indicate that the performance of Multi-Agent Soft Actor-Critic (MASAC) is better than that of other MADRL methods by a large margin. UAV swarm employing MASAC can learn more effective policies, and obtain much higher hit rate and win rate. Simulations under different swarm sizes and UAV physical parameters are also performed, which implies that MASAC owns a well generalization effect. Furthermore, the practicability and convergence of MASAC are addressed by investigating the loss value of Q-value networks with respect to individual UAV, the results demonstrate that MASAC is of good practicability and the Nash equilibrium of the UAV swarm non-cooperative game under incomplete information can be reached.
© 2024 Chinese Society of Aeronautics and Astronautics. Production and hosting by Elsevier Ltd. All rights reserved. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

The fast increase in the use of Unmanned Aerial Vehicles (UAVs) in recent years leads to their increasing important role in the military field. [1] UAV swarm system is a typical complex multi-agent game system, where a number of homogeneous or heterogeneous intelligent UAVs are composed into a powerful cluster, and UAVs relying on local sensing and reactive

* Corresponding author.
E-mail address: hchchina@sina.com (C. HONG).
Peer review under responsibility of Editorial Committee of CJA.



https://doi.org/10.1016/j.cja.2024.03.030
1000-9361 © 2024 Chinese Society of Aeronautics and Astronautics. Production and hosting by Elsevier Ltd. All rights reserved.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

behaviors can interact with the environment such that a global behavior emerges from the interactions. Communicating by wireless networks, UAV swarm possesses the capabilities of perception, recognition, interactive learning, collaborative planning, autonomous decision-making and so on. [2–4] The study of UAV swarm has given rise to great achievements in many fields, such as cooperative search, [5] air combat decision, [6] cooperative aerial interception, [7] target detection, [8] task planning, [9] etc.

Some methods have been proposed to solve non-cooperative game under incomplete information. Burch et al. [10] proposed the first technology to decompose incomplete information games into sub-games that can be solved independently, while preserving the optimality guarantee of the entire game solution. Rowland et al. [11] introduced different sampling algorithms to adaptively allocate samples to the agents that can best provide ranking information, and spread the uncertainty in the game payoff to the uncertainty in the ranking of agents. These contributions make it possible to conduct principled evaluation of agents under incomplete information. Duan et al. [12]presented a game theoretic approach with Predator-Prey Particle Swarm Optimization (PP-PSO) to solve the dynamic task assignment problem for multiple Unmanned Combat Aerial Vehicles (UCAVs), where the military operation is modeled as a two-player game. And the optimal assignment scheme of each stage is regarded as a mixed Nash equilibrium solved by the PP-PSO. Huang et al. [13] regarded the air combat game as a Markov process, and calculated the air combat situation via Bayesian inference theory. Then the whole air combat maneuver decision process is modeled by using the moving horizon optimization strategy. Li et al. [14] decomposed the dynamic game process into different decision-making stages, and modeled the current decision-making stage as a zero-sum game model. In the model, an uncertain payoff matrix is used to describe the incomplete strategies. Based on the maximum-minimum strategy in matrix games, they proposed a solution method of the established game under an incomplete strategy set.

Comparing with traditional paradigms, Reinforcement Learning (RL) does not require any data to be given in advance, but rather obtains learning information and updates model parameters by receiving the reward (feedback) from the environment to the action. Moreover, complex RL algorithms have general intelligence to solve complex problems to some extent, and can reach human level in Go and video games. Actually, owning the flexible interactivity between agents and environment, RL paradigm is of natural advantages in complex dynamical game environment. [15] One of the pioneering works is the Deep Q-Learning (DQN), which solves the problem that RL cannot be applied to continuous state spaces. [16] Subsequently, many different variants of DQN have been proposed, including dueling DQN, [17] double DQN, [18] Dueling Double DQN (D3QN), [19] Recurrent Replay Distributed DQN (R2D2), [20] Never Give Up (NGU), [21] agent57, [22] DQNreg [23] and so on. REINFORCE [24] is a classic algorithm using the Actor-Critic (AC) framework, where value-based and policy-based methods are elaborately combined. [25] The Advantage Actor-Critic (A2C) is proposed to reduce variance, and the Asynchronous Advantage Actor-Critic (A3C) is put forward to accelerate the training process. [26] Based on the Deterministic Policy Gradient (DPG), [27] Lillicrap et al. [28] presented an approach named the deep deterministic policy gradient (DDPG), [28] which is a model-free, off-policy actor-critic algorithm using deep function approximators that can learn policies in high-dimensional, continuous action spaces. Haarnoja et al. proposed an off-policy maximum entropy actor-critic algorithm, which is named Soft Actor-Critic (SAC). It provides for both sample-efficient learning and stability, and can be extended readily to very complex, high-dimensional single agent tasks. [29]

Compared with the single agent RL, Multi-Agent Reinforcement Learning (MARL) is more complex and hard to converge. In the MARL environment, from the perspective of an individual agent, the behavioral decision-making of other agents is often unknown. Each agent only executes its own policy based on its incomplete observation information, and hardly adapts the dynamic changes of the overall environment, resulting in the non-stationarity of the environment. The most intuitive way to solve the problem is to view all agents as a big integrated agent, so the Markov Decision Process (MDP) can be used directly. Unfortunately, this will cause an exponential explosion of the action space when the number of agents is large.

Actually, in some environments of MARL, agent may learn in a virtual or simulated environment, in which extra observation information can be collected. Noticed this phenomenon, Lowe et al. [30] proposed an effective paradigm called Centralized Training with Decentralized Execution (CTDE). Thereafter, many CTDE-based algorithms have been developed, such as the Multi-Agent Deep Deterministic Policy Gradient (MADDPG), [30] the Multi-Agent Proximal Policy Optimization (MAPPO). [31] However, the credit assignment of multiple agents is still a nasty problem, which can cause greater variance in CTDE. To reduce variance and address the credit assignment issue, Foerster et al. proposed the Counterfactual Policy Gradient (COMA) algorithm. [32]

So far, RL has been widely used in the domains of UAV. [33] Liu and Ma established an UAV confrontation game model by combining DQN with the maximum-minimum algorithm. [34] Toghiani-Rizi et al. applied Deep Reinforcement Learning (DRL) to model UAV tactical behaviors. [35] Using DDPG, Yang et al. explored autonomous maneuver decision-making method with respect to UAV air combat. [36] Kong et al. put forward a maneuver strategy of UAV autonomous aerial combat by employing DDPG. [37] Qiu et al. proposed a one-to-one air-combat maneuver strategy via the improved TD3 algorithm. [38–39] Cheng and Song [40] constructed an UAV attack and defense model based on SAC.

Meanwhile, MARL has been extensively applied in UAV swarm scenes. [41–46] To resolve the issues of dimensional curse as well as environmental non-stationarity, via DRL, an improved proximal policy optimization method is proposed to solve UAV swarm game problem. [47] Based on DRL, a multi-aircraft cooperative air combat method is put forward. [48] Combining MADDPG with transfer learning and self-play, an effective UAV swarm confrontation game model is constructed. [49] Huang et al. use DDPG and MADDPG to tackle the multi-agent defense and attack tasks, the results indicate that MADDPG achieves superior performance than other DRL-based methods. [50] By using the curriculum learning and MADDPG, the learning ability of UAV swarm can be enhanced in the high-dimensional environment. [51]

Recently, the advantage of RL method has been extensively used in solving UAV air combat problems. Gong et al. [52]

proposed a multi-UAV cooperative maneuvering decision-making approach based on Multi-Agent Deep Reinforcement Learning (MADRL) theory to address the problem of multi-UAV cooperative air combat decision-making. Cao et al. [53] used the Double Deep Q Network (DDQN) algorithm to train the agent by the interaction with the environment, and combined with Minimax algorithm in stochastic game theory to obtain the optimal maneuver decision of Unmanned Combat Aerial Vehicle (UCAV). Aiming at the intelligent decision-making problem faced in the process of UAV air combat, Zhang et al. [54] established a multi-dimensional decision-making model based on the hierarchical reinforcement learning architecture, and the result shows a good countermeasure effect in the multi-dimensional decision-making problem of UAV air combat. Considering the problem of UAV's autonomous decision-making in close air combat, Qiu et al. [55] proposed an autonomous guidance method based on Proximal Policy Optimization (PPO) algorithm; the authors also established a three-degree-of-freedom model and reshaped the rewards corresponding to the distance, angle, speed and mission constraint.

Most of previous MARL works applied to UAV swarm combat are usually limited to simple scenes and the state and action spaces are often discrete. However, in real UAV swarm air-to-air combat scenarios, for each UAV, its action space is continuous due to the turn angle or velocity, and its observation is incomplete because of the sensors. Inspired by the idea of maximum entropy and CTDE training framework, we introduce MADRL into UAV swarm game under incomplete information and construct a MADRL environment with continuous and infinite action space. The main contributions of this paper include:

(1) Considering the flight dynamics of UAVs and the partially observable Markov game, the state space and action space of MARL environment are exquisitely constructed to adapt the real features of UAV swarm air-to-air combat.
(2) Based on the Multi-Agent Particle Environment (MPE), we build an UAV swarm non-cooperative game model incorporating MADRL methods.
(3) Some recently popular MADRL methods are investigated extensively in the UAV swarm non-cooperative game model. This shows that Multi-Agent Soft Actor-Critic (MASAC) outperforms other MADRL methods by a substantial margin.
(4) The generalization effect of MASAC is investigated accordingly, which indicates that MASAC is of a high generalization ability with respect to swarm sizes and physical parameters.
(5) The practicability and convergence of MASAC are explored by investigating the loss value of Q-value networks, the results show that MASAC is practicable and the Nash equilibrium of the UAV swarm non-cooperative game under incomplete information can be reached.

The paper is organized as follows. In the next section we demonstrate the MADRL-based UAV swarm non-cooperative game model under incomplete information in detail. In Section 3, simulation results and discussion are provided. Finally, the work is summarized in Section 4.

2. Model

2.1. UAV swarm combat environment

The flight dynamics of each UAV is denoted as:[49,56–57]

$$\begin{cases} \phi_t = \phi_{t-1} + r_\phi \mathrm{d}t, \quad -30 < \phi_{t-1},\ \phi_t < 30 \\ r_\varphi = 9.81 \cdot m/F \cdot \mathrm{d}t \cdot \tan\phi_t \\ \varphi_t = \varphi_{t-1} + r_\varphi \mathrm{d}t, \quad -180 < \varphi_{t-1},\ \varphi_t < 180 \\ v_x = \sin\varphi_t \cdot F/m \\ v_y = \cos\varphi_t \cdot F/m \\ x_t = x_{t-1} + v_x \mathrm{d}t \\ y_t = y_{t-1} + v_y \mathrm{d}t \end{cases} \tag{1}$$

where ϕ_t denotes UAV's roll angle at time step t, and r_ϕ is the roll angular velocity; F is the driving force, and m is UAV's mass; φ_t denotes the course angle at time step t, and r_φ is the course angular velocity; (x_t, y_t) denotes the coordinates of the UAV at time step t, and $\mathrm{d}t$ is the differential variable of time.

The combatant situation between the attacker (UAV i) and the defender (UAV j) is illustrated in Fig. 1, where a relative geometry state of the attacker and the defender is displayed, and this illustration is used to represent a simple snapshot of UAV air-to-air defense attack scenario. The attacking zone is in front of the UAV, covering a sector of θ_{ac}. The unprotected zone is a θ_{de} sector behind the UAV. The relation of the attacker-target pair can be characterized by a quaternion $\boldsymbol{s}(t) = \left[\omega(t), d_{ij}, \psi^i_{\mathrm{ac}}, \psi^j_{\mathrm{de}}\right]^{\mathrm{T}}$, which is computed by [49]

$$\omega(t) = \left(\omega_x(t), \omega_y(t)\right) = \left(x_i - x_j, y_i - y_j\right) \tag{2}$$

$$d_{ij} = \sqrt{\left(x_j - x_i\right)^2 + \left(y_j - y_i\right)^2} \tag{3}$$

$$\psi^i_{\mathrm{ac}} = \arccos \frac{x_i \omega_x(t) + y_i \omega_y(t))}{d_{ij}\sqrt{x_i^2 + y_i^2}} \tag{4}$$

$$\psi^j_{\mathrm{de}} = \arccos \frac{-x_j \omega_x(t) - y_j \omega_y(t)}{d_{ij}\sqrt{x_j^2 + y_j^2}} \tag{5}$$

where d_{ij} is the Euclidean distance of the attacker-defender pair, and ψ^i_{ac} denotes the attacking angle of the attacker while ψ^i_{de} is the defending angle of the defender.

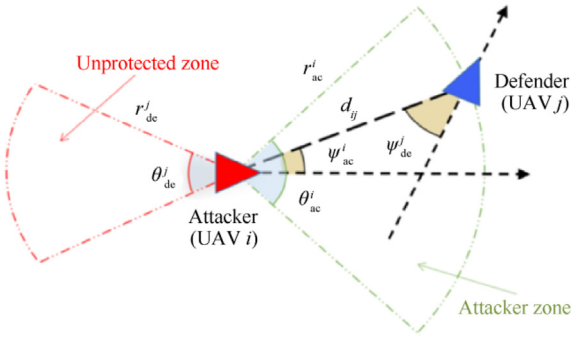


Fig. 1 An illustration of the combatant situation between an attacker (UAV i) and a defender (UAV j).

Here, UAV j can be killed by UAV i when three constraints are satisfied: (A) the distance of the attacker-defender pair neither greater than the radius of the attacking zone nor greater than the radius of the unprotected zone; (B) the defender locates in i's attacking zone and (C) the attacker in j's unprotected zone. Such constraints can be denoted as

$$d_{ij} \leqslant r^i_{\text{ac}} \text{ and } d_{ij} \leqslant r^j_{\text{de}} \tag{6}$$

$$\psi^i_{\text{ac}} \cdot 180/\pi \leqslant \theta^i_{\text{ac}}/2 \tag{7}$$

$$\psi^j_{\text{de}} \cdot 180/\pi \leqslant 180 - \theta^j_{\text{de}}/2 \tag{8}$$

where r^i_{ac} denotes the radius of UAV i's attacking zone, and r^j_{de} is the radius of UAV j's unprotected zone.

2.2. UAV swarm non-cooperative game under inco-mplete information

For single UAV's learning environment, UAV learns its policy via interacting with the environment, which is usually modeled as a MDP, and the goal of the UAV is to maximize its accumulated discounted reward. In the case of UAV swarm environment, there exist cooperation, competition or mixed relation among UAVs. Each UAV need to consider and interact with not only the environment but also other UAVs. Therefore, MDPs involving multiple UAVs can be naturally modeled as a Stochastic Game (SG), [58] also known as a Markov Game (MG), [59] which plays as a framework that allows simultaneous actions of UAVs. Since individual UAV's observation is partial or incomplete, UAV swarm air combat game can be described as a Partially Observable Markov Game (POMG), or a Partially Observable Stochastic Game (POSG). [59]

POMG is one of the most general classes of games. A POMG of UAV swarm with N UAVs can be denoted by a set of states S describing the possible configurations of all UAVs, a set of actions $A_1, A_2, ..., A_N$ as well as a set of observations $O_1, O_2, ..., O_N$. In such partially observable environments, S is incomplete for individual UAV and each UAV receives only a private observation that is correlated with S. The goal of each UAV is to obtain the optimal policy with respect to the POMG. At each time step, the environment exhibits a state, and each UAV executes its action, simultaneously with all other UAVs. The joint action of all UAVs makes the environment transition into the next state, and the environment offers an immediate reward to each UAV.

To select actions, UAV i uses a stochastic policy $\pi_i : O_i \times A_i \mapsto [0, 1]$, which generates the next state according to the state transition function $\Gamma : S \times A_1 \times ... \times A_N \mapsto S'$. UAV i receives rewards $r_i : S \times A_i \mapsto R$ as a function of the state and its action, and obtains a private observation $o_i : S \mapsto O_i$ correlated with the state. UAV i aims to maximize its own total expected return $R_i = \sum_{t=0}^{\infty} \gamma^t r_i^t$, where $\gamma \in [0, 1]$ is a discount factor who reflects the relative importance between the long-term reward and the current reward.

Actually, game theory is mainly a decision-making theory in a conflict environment, which focuses on the behavior interaction between players. It is a mathematical theory and method to study the optimal strategy solution under the competitive or cooperative relationship. In the game, players interact through the selection of actions to maximize their own payoffs. MARL is, in essence, RL applied to multi-agent game models to learn optimal policies for the agents. The central goal of MARL research is to develop learning algorithms which can learn agent policies that robustly converge to a particular equilibrium-type solution. The significance of the combination of game theory and MARL lies in their complementary advantages. Game theory provides an easy-to-handle solution concept to describe the learning results of multi-agent systems, while MARL algorithms provide convergent learning schemes that can achieve stability and rational equilibrium in the process of sequential decision-making. Algorithms that combine game theory with MARL have boomed and achieved many surprising results in recent years, especially for solving multi-agent game problems under incomplete information. [60–61] Many types of equilibrium solution concepts exist for MARL, among which the most famous is probably the Nash Equilibrium (NE) in non-cooperative game theory. [62] For a MARL process, Nash equilibrium is a connection strategy of all agents. At the Nash equilibrium, all agents cannot obtain greater rewards without changing their own strategies. Here the word "non-cooperative" does not mean agents cannot collaborate to complete a task, it merely means that each agent maximizes its own reward independently and that agents cannot group into coalitions to make collective decisions. [63]

The learning goals in MARL are multi-dimensional. Bowling and Veloso [64–65] proposed to classify the goals of the learning task into two types: rationality and convergence. Rationality ensures an agent takes the best possible response to the opponents, while convergence ensures the learning dynamics eventually lead to a stable policy, then reaching both rationality and convergence gives rise to reaching the Nash equilibrium of the multi-agent non-cooperative game. [64–65] In the UAV swarm non-cooperative game model, each UAV is regarded as an agent, and thus each UAV maximizes its own reward independently in the MARL environment. Obviously, this is a dynamical Markov game model under incomplete information due to each UAV's partial or private observations.

An influential article by Shoham et al. [66] proposed several distinct agendas which have been pursued in MARL research. One of the most important agendas is computational agenda who describes that MARL is an approach to compute solutions for game models. A solution consists of a collection of decision policies for the agents that satisfy certain features such as Nash equilibrium. Once computed, a solution could be deployed in an application of the game to control the agents. Therefore, in the computational agenda, MARL methods compete with other traditional direct methods to compute game solutions. Such direct methods may be more efficient than MARL methods for certain types of games, but they typically require full knowledge of the game. On the contrary, most MARL methods can learn without full knowledge of the game. Due to the incomplete information in the UAV swarm air combat game model, MARL is a feasible paradigm for obtaining the game solution. Consequently, for the UAV swarm air-to-air combat environment, we use POMG to model and MARL method to solve. Different from single agent RL, MARL needs to determine the corresponding solution concept according to the payoff relationship of UAVs, and then use MARL algorithm to solve the corresponding target, namely, reach the Nash equilibrium. [67] In other words, MARL methods are applied to make the rewards (payoffs) approach a con-

vergence, [68] and regarded as calculation methods in solving the Nash equilibrium of the UAV swarm air combat game under incomplete information.

For an incomplete information game, obtaining belief states can be provided by the Bayesian approach known as partially observable MGs, or POMGs. In the UAV swarm POMG, since an UAV's observation only provides partial information about the current state of the environment, it is typically not possible to choose optimal actions based only on the current observation. Therefore, on the strength of the history of observations saved in the experience replay buffer, UAVs can maintain estimates of the possible current states, and such estimates for UAV i is denoted as a belief state b_i^t, which is a probability distribution over the states $\boldsymbol{s} \in S$ that the environment may be exhibited at time t. Based on the CTDE paradigm, the critic network can receive global state-action pairs from all UAVs during the process of centralized training. Consequently, given the initial state distribution $Pr_0 : S \mapsto [0,1]$, a new observation O_i^{t+1}, and the past joint action $\boldsymbol{a}^t$ of all UAVs, we can update the belief state by computing a Bayesian posterior distribution,

$$b_i^{t+1}(\boldsymbol{s}') = \eta \sum_{\boldsymbol{s} \in S} b_i^t(\boldsymbol{s}) \tau(\boldsymbol{s}'|\boldsymbol{s}, \boldsymbol{a}^t) O_i\left(\boldsymbol{o}_i^{t+1}|\boldsymbol{a}^t, \boldsymbol{s}'\right)$$

where $b_0^i = Pr_0$, and η is a normalisation constant. [69] Since the Bayesian approach can provide an elegant approach to action-selection as a function of the uncertainty in learning, [70] in the MADRL-based UAV swarm non-cooperative game under incomplete information, the Nash equilibrium can be characterized to maximize the expected payoff of all UAVs.

Under multi-agent non-cooperative game, the convergence criterion is Nash equilibrium. When formulating strategies, an intelligent agent should consider the strategies of other players. In the case of Nash equilibrium, every intelligent agent is responding to the policies of other players in the optimal way. No one has the motivation to change their own strategy alone, as changing the strategy will not increase their own profits, thus achieving a state of balance. This equilibrium state is considered as convergence, because no intelligent agent can find a better solution. In the UAV swarm non-cooperative game, Nash equilibrium is described as follows.

Nash equilibrium:

In an UAV swarm non-cooperative game with N UAVs, when all the other UAVs do not change their own strategy, an UAV i cannot change its strategy alone to make its expected return $J^i\left(\theta^1, \theta^2, \cdots, \theta^N\right)$ larger. If the average return of all UAVs does not change, it can be considered that Nash equilibrium has been reached.

2.2.1. State space

In the UAV swarm non-cooperative game model, the state space describes the observations that the UAV receives from the environment. Here, the electronic interference among combat UAVs will be taken into account, and the interference function of UAV i is denoted by

$$\delta_{i,t} = \begin{cases} 1, & \text{Interfere opponents} \\ 0, & \text{Otherwise} \end{cases} \tag{9}$$

where $\delta_{i,t} = 1$ means that UAV i interferes its opponents at time step t.

For UAV i, there are three kinds of information: (A) the information of self-state; (B) the observation information of i's teammates and (C) the observation information of i's opponents. Therefore, the state space of UAV i at time step t can be defined as

$$\boldsymbol{o}_{i,t} = [\boldsymbol{s}_{i,t}, \boldsymbol{f}_{i,t}, \boldsymbol{e}_{i,t}] \tag{10}$$

where $\boldsymbol{s}_{i,t}$ represents the self-state feature vector of UAV i, $\boldsymbol{f}_{i,t}$ denotes the observation feature vector of i's teammates, and $\boldsymbol{e}_{i,t}$ is the observation feature vector of i's opponents. The self-state feature vector $\boldsymbol{s}_{i,t}$ is given by

$$\boldsymbol{s}_{i,t} = \left[x_{i,t}, y_{i,t}, v_{x,i}, v_{y,i}, \theta_{i,t}, g_{i,t}, \boldsymbol{a}_{i,t-1}\right] \tag{11}$$

where $\left(x_{i,t}, y_{i,t}\right)$ is the relative coordinates between UAV i and the origin of the two-dimensional Cartesian coordinate system, $\left(v_{x,i}, v_{y,i}\right)$ is the velocity vector of UAV i, and $\theta_{i,t}$ is UAV i's course angle at time step t. The remained interference number of UAV i is given by $g_{i,t} = g_{i,t-1} - \delta_{i,t-1}$, where $\delta_{i,t-1}$ is i's interference function at time step $t-1$. And $\boldsymbol{a}_{i,t-1}$ is the action of UAV i at time step $t-1$. The observation feature vector of i's teammates can be defined as

$$\boldsymbol{f}_{i,t} = [\hat{\boldsymbol{s}}_{1,t}, \hat{\boldsymbol{s}}_{2,t}, \cdots, \hat{\boldsymbol{s}}_{i-1,t}, \hat{\boldsymbol{s}}_{i+1,t}, \cdots, \hat{\boldsymbol{s}}_{N,t}] \tag{12}$$

where $\hat{\boldsymbol{s}}_{j,t}$ is the feature of the j-th teammate, $j = 1, 2, ..., N, j \neq i$, and it can be denoted as

$$\hat{\boldsymbol{s}}_{j,t} = \left[d_{i,j,t}, \theta_{j,t}, v_{x,j,t}, v_{y,j,t}\right] \tag{13}$$

where $d_{i,j,t}$ represents the distance between UAV i and UAV j at time step t, which can be computed by

$$d_{i,j,t} = \|p_{i,t} - p_{j,t}\|_2 = \sqrt{\left(x_{i,t} - x_{j,t}\right)^2 + \left(y_{i,t} - y_{j,t}\right)^2}$$

The observation feature of i's opponents is given by

$$\boldsymbol{e}_{i,t} = [\hat{\boldsymbol{s}}_{1,t}, \hat{\boldsymbol{s}}_{2,t}, ..., \hat{\boldsymbol{s}}_{k,t}, ..., \hat{\boldsymbol{s}}_{N^-,t}] \tag{14}$$

where $\hat{\boldsymbol{s}}_{k,t}$ represents the feature of the k-th opponent, $k = 1, 2, ..., N^-$, N^- is the number of opponents. And $\hat{s}_{k,t}$ can be denoted as

$$\hat{\boldsymbol{s}}_{k,t} = \left[d_{i,k,t}, \theta_{k,t}, v_{x,k,t}, v_{y,k,t}, g_{k,t}\right]$$

where $\theta_{k,t}$ and $g_{k,t}$ denote the course angle and the remained interference number of the k-th opponent, respectively.

2.2.2. Action space

In the UAV swarm non-cooperative game model, the action space describes the allowed actions that the UAV interacts with the RL environment, and the optimal decision-making learned by the UAV is reflected through actions. Here, each UAV owns a continuous and infinite action space, in which the velocity vector for each one can be computed by

$$\begin{cases} v_{x,i,t} = v_{x,i,t-1} + \sin \varphi_{i,t} \cdot F_{i,t}/m_i \cdot \mathrm{d}t \\ v_{y,i,t} = v_{y,i,t-1} + \cos \varphi_{i,t} \cdot F_{i,t}/m_i \cdot \mathrm{d}t \\ \phi_{i,t} = \phi_{i,t-1} + r_{\phi,i,t} \cdot \mathrm{d}t \end{cases} \tag{15}$$

where $F_{i,t}$ is UAV i's driving force at time step t, and m_i is the mass of UAV i; $\varphi_{i,t}$, $\phi_{i,t}$ and $r_{\phi,i,t}$ are respectively the course angle, the roll angle and the roll angular velocity of UAV i at time step t; dt is the differential variable of time. It is noteworthy that all UAVs can fly with a variable speed in an arbitrary direction due to the continuous action space.

In our UAV swarm non-cooperative game model, the action space of UAV i at time step t can be defined as $\boldsymbol{A}_i = [F_{i,t}, r_{\varphi,i,t}, \delta_{i,t}]$, where $\delta_{i,t}$ is UAV i's interference function defined in Eq. (9). The parameters of the action space are linked to the flight dynamics denoted by Eq. (1) to interact with the RL environment of the game model. Therefore, the joint action space of N UAVs can be obtained by $\boldsymbol{A} = \boldsymbol{A}_1 \times \boldsymbol{A}_2 \times ... \times \boldsymbol{A}_N$.

2.2.3. Reward function

The reward function is the incentive mechanism for an UAV to learn a better action, which can illustrate the performance when an action is executed under a specific state. To make the employed MADRL methods more advantageous in solving UAV swarm combat tasks, the reward function consists of five items: the attack reward, the threat reward, the interference reward, the disturbed reward and the distance reward.

Note that UAV i should be rewarded when it attacks its opponents, the attack reward function of UAV i at time step t can be denoted as

$$\begin{cases} r_{1,i,t} = \lambda_{1,i} \cdot \text{sum}(\boldsymbol{w}_{i,t}) \\ \boldsymbol{w}_{i,t} = [w_1, w_2, \cdots, w_{N^-}] \end{cases} \tag{16}$$

where N^- is the number of UAV i's opponents, $\lambda_{1,i}$ denotes the weight of the attack reward function, and $\boldsymbol{w}_{i,t}$ is the one-hot encoding vector of the ID of the opponents attacked by UAV i at time step t.

To protect UAV i from threating, the threat reward function is given by

$$\begin{cases} r_{2,i,t} = -\lambda_{2,i} \cdot \text{sum}(\boldsymbol{u}_{i,t}) \\ \boldsymbol{u}_{i,t} = [u_1, u_2, \cdots, u_{N^-}] \end{cases} \tag{17}$$

where $\lambda_{2,i}$ is the weight of the threat reward function, and $\boldsymbol{u}_{i,t}$ represents the one-hot encoding vector of the ID of the opponents threatening UAV i at time step t. Contrary to the attack reward, one can see from Eq. (17) that UAV i will receive a negative reward as a penalty when it is threatened by its opponents.

To encourage UAV i to release the interference, the interference reward function is denoted as

$$\begin{cases} r_{3,i,t} = \lambda_{3,i} \cdot \text{sum}(\boldsymbol{b}_{i,t}) \\ \boldsymbol{b}_{i,t} = [b_1, \ b_2, \ \cdots, \ b_{N^-}] \end{cases} \tag{18}$$

where $\lambda_{3,i}$ is the weight of the interference reward function, and $\boldsymbol{b}_{i,t}$ represents the one-hot encoding vector of the ID of the opponents disturbed by UAV i at time step t.

On the other hand, when UAV i is disturbed by its opponents, it will receive a negative reward as a penalty. The disturbed reward function of UAV i is represented as

$$\begin{cases} r_{4,i,t} = -\lambda_{4,i} \cdot \text{sum}(\boldsymbol{c}_{i,t}) \\ \boldsymbol{c}_{i,t} = [c_1, \ c_2, \ \cdots, \ c_{N^-}] \end{cases} \tag{19}$$

where $\lambda_{4,i}$ is the weight of the disturbed reward function, and $\boldsymbol{c}_{i,t}$ represents the one-hot encoding vector of the ID of the opponents releasing the interference to UAV i at time step t.

To avoid the problem of sparse reward, the distance reward function is introduced. Note that the shorter the distance of two rival UAVs is, the easier the killing of the target is. Hence the reward of UAV i should be inversely proportional to the minimal distance between UAV i and UAV j, the distance reward function is given by

$$r_{5,i,t} = -\lambda_{5,i} \cdot \min\left(\sqrt{(x_{i,t} - x_{j,t})^2 + (y_{i,t} - y_{j,t})^2}, \ j \in \Gamma_{N^-}\right) \tag{20}$$

where Γ_{N^-} is the set of opponents of UAV i, and $\lambda_{5,i}$ represents the weight of the distance reward function. It is noteworthy that only the nearest opponent of UAV i is considered, avoiding that UAV i constantly swings between different opponents.

To sum up, the full reward function of UAV i at time step t can be denoted as

$$r_{i,t} = r_{1,i,t} + r_{2,i,t} + r_{3,i,t} + r_{4,i,t} + r_{5,i,t} \tag{21}$$

One can see from Eq. (20), the reward function is a comprehensive reward consisting of five sub rewards: the attack reward, the threat reward, the interference reward, the disturbed reward and the distance reward. By using the weight parameters $(\lambda_{1,i}, \lambda_{2,i}, \lambda_{3,i}, \lambda_{4,i}, \lambda_{5,i})$, the reward function is actually a weighted sum of five sub rewards. Consequently, the reward function in our RL environment can be described as a comprehensive payoff with respect to UAV's current combat situation.

Generally, multi-agent interactions are characterized by game models. It is naturally that the UAV swarm air combat can be described by a game model. Here the payoff of player (UAV) i is not only related to the policy π_i it adopts, but also affected by the joint policy π_{-i} of other players (UAVs). Therefore, an important problem is to determine the optimization goal of the UAV swarm game system, i.e., the solution of the game. Next, we will elaborate in detail the MADRL methods for computing the solution of the UAV swarm non-cooperative game.

2.3. Multi-agent deep reinforcement learning methods

2.3.1. Decentralized deep deterministic policy gradient (Dec-DDPG)

Comparing with centralized learning that iterates through fixed rules on each UAV, decentralized RL can solve the problem of action space exploding caused by the increase of the number of UAVs, converging UAVs' dynamics to a certain equilibrium. Dec-DDPG is one of the important decentralized RL methods.

The deterministic strategy is of many advantages such as less learning data and higher efficiency than the stochastic strategy. As a successful extension of DPG, [27] DDPG is a particularly popular model-free, off-policy actor-critic RL method, where the actor and critic are approximated with deep function approximators that can learn policies in high-dimensional, continuous action spaces. Dec-DDPG is a decentralized MADRL method based on DDPG, where polices and evaluations for each UAV are only determined by its own state and action. For UAV i, the loss function of the critic network is given by

$$\begin{cases} L(\mu_i) = \mathbb{E}_{\boldsymbol{o}_i, \boldsymbol{a}_i, r_i, \boldsymbol{o}'_i}\left[(Q_i(\boldsymbol{o}_i, \boldsymbol{a}_i; \mu_i) - y)^2\right] \\ y = r_i + \gamma \bar{Q}_i(\boldsymbol{o}'_i, \boldsymbol{a}'_i; \bar{\mu}_i)|_{\boldsymbol{a}'_i = \pi_i(\boldsymbol{o}'_i; \bar{\theta}_i)} \end{cases}$$

where $\bar{Q}_i$ denotes the critic target network of Q_iTo help the learning stability, the target network parameter $\bar{\mu}_i$ is soft updated periodically with the most recent μ_i with respect to the critic network. On the other, $\bar{\pi}_i$ represents the actor target network, and $\bar{\theta}_i$ is soft updated periodically with the most recent θ_i of the actor network.

The objective function of the actor network is denoted as $J(\theta_i) = \mathbb{E}_{(\boldsymbol{o}_i \sim \rho_\theta)}[R_i]$, and its gradient can be derived as

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{\boldsymbol{o}_i \sim \rho_\theta}[\nabla_{\theta_i} \pi_i(\boldsymbol{o}_i;\theta_i) \nabla_{\boldsymbol{a}_{i,t}} Q(\boldsymbol{o}_i, \boldsymbol{a}_i)|_{\boldsymbol{a}_i = \pi_i(\boldsymbol{o}_i;\theta_i)}]$$

2.3.2. Decentralized soft actor-critic (Dec-SAC)

Soft Actor-Critic (SAC) is an important stochastic policy gradient method and has shown good performance on many single agent learning domains. [28–29] SAC uses the maximum entropy mechanism and possesses model-free as well as off-policy properties, where an Actor-Critic (AC) framework with policy and Q-value networks is adopted, and recently experienced transitions in the replay buffer are sampled. SAC combines off-policy and actor-critic training with a stochastic actor, and further aims to maximize the entropy of the actor. Dec-SAC is a SAC-based decentralized version who works in the multi-agent environment, which can be described as

$\pi_i^* = \arg\max \sum_t \mathbb{E}_{\boldsymbol{o}_i \sim \rho_\theta, \boldsymbol{a}_i \sim \pi_i}[R_i(\boldsymbol{o}_i, \boldsymbol{a}_i) + \alpha \mathscr{H}(\pi_i(\cdot|\boldsymbol{o}_i))]$ where the temperature parameter α adjusts the relative importance of the entropy term against the reward. Here the entropy is defined as

$$\mathscr{H}(\pi_i(\cdot|\boldsymbol{o}_i)) = \mathbb{E}[-\log(\pi_i(\cdot|\boldsymbol{o}_i))]$$

The objective function of the actor network is denoted as

$$J(\theta_i) = \mathbb{E}_{\boldsymbol{o}_i \sim D}\left[\mathbb{E}_{\boldsymbol{a}_i \sim \pi_{\theta_i}}\left[\min_{j \in \{1,2\}} Q_{i,j}^{\text{soft}}(\boldsymbol{o}_i, \boldsymbol{a}_i; \mu_{i,j}) - \alpha \log \pi(\boldsymbol{a}_i|\boldsymbol{o}_i;\theta_i)\right]\right]$$

where D is the experience replay buffer who records the trajectory data. Bootstrapping is used for updating the soft Q-function, which is given by

$Q_{i,j}^{\text{soft}}(\boldsymbol{o}_i, \boldsymbol{a}_i; \mu_{i,j}) = \mathbb{E}_{\boldsymbol{o}_i \sim \rho_\theta, \boldsymbol{a}_i \sim \pi_\theta}[r_i - \alpha \log \pi_i(\cdot|\boldsymbol{o}_i)]$, $j = 1, 2$ The loss function of the twin critic network is denoted as follows.

$$\begin{cases} L_j(\mu_i) = \mathbb{E}_{\boldsymbol{o}_i, \boldsymbol{a}_i, r_i, \boldsymbol{o}_i'}\left[\left(Q_{i,j}^{\text{soft}}(\boldsymbol{o}_i, \boldsymbol{a}_i; \mu_{i,j}) - y\right)^2\right] \\ y = r_i + \gamma \min_{j \in \{1,2\}} \bar{Q}_{i,j}^{\text{soft}}(\boldsymbol{o}_i, \boldsymbol{a}_i'; \bar{\mu}_{i,j}) \end{cases}$$

where $\bar{Q}_{i,j}^{\text{soft}}$ denotes the twin critic target network of $Q_{i,j}^{\text{soft}}$, and its network parameter $\bar{\mu}_{i,j}$ is periodically soft updated with the most recent $\mu_{i,j}$ with respect to the critic network.

2.3.3. Multi-agent deep deterministic policy gradient

MADDPG is an extension version of DDPG, which is based on AC framework and has been used in many multi-agent decision-making tasks. CTDE paradigm is put forward to improve the efficiency of MADDPG. [30] Accordingly, for MADDPG, during the process of centralized training, the critic network receives global state-action pairs from all agents, while the actor network only relies individual observations during the decentralized execution process.

Here, the global observation state vector $\boldsymbol{o} = [\boldsymbol{o}_1, \boldsymbol{o}_2, ..., \boldsymbol{o}_N]$, the global action vector $\boldsymbol{a} = [\boldsymbol{a}_1, \boldsymbol{a}_2, ..., \boldsymbol{a}_N]$, the policy is parameterized by $\theta = \{\theta_1, \theta_2, ..., \theta_N\}$, and $\pi = \{\pi_1, \pi_2, ..., \pi_N\}$ denotes the set of all policies. The gradient of the actor network is given by

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{\boldsymbol{o}, \boldsymbol{a} \sim D}\left[\nabla_{\theta_i} \pi_i(\boldsymbol{o}_i;\theta_i) \nabla_{\boldsymbol{a}_i} Q_i^\pi(\boldsymbol{o}, \boldsymbol{a}_i, \boldsymbol{a}_{-i}; \mu_i)|_{\boldsymbol{a}_i = \pi_i(\boldsymbol{o}_i;\theta_i)}\right]$$

where $\boldsymbol{a}_{-i} = [\boldsymbol{a}_1, ..., \boldsymbol{a}_{i-1}, \boldsymbol{a}_{i+1}, ..., \boldsymbol{a}_{-N}]$ is the action vector for all agents except agent i, and D records the recently experienced trajectories of all agents. The centralized state-action pair value function $Q_i^\pi(\boldsymbol{o}, \boldsymbol{a}_i, \boldsymbol{a}_{-i})$ is the output of the critic network, while the global observation states and the actions of all agents are inputs of the critic network. The loss function of the critic network is defined as follows.

$$\begin{cases} L(\mu_i) = \mathbb{E}_{\boldsymbol{o}, \boldsymbol{a}, r_i, \boldsymbol{o}'}\left[\left(Q_i^\pi(\boldsymbol{o}, \boldsymbol{a}; \mu_i) - y\right)^2\right] \\ y = r_i + \gamma \bar{Q}_i^\pi(\boldsymbol{o}', \boldsymbol{a}'; \bar{\mu}_i)|_{\boldsymbol{a}'_j = \pi_j(\boldsymbol{o}'_j; \bar{\theta}_j)} \end{cases}$$

where $\bar{Q}_i^\pi$ represents the critic target network of Q_i^π, and its network parameter $\bar{\mu}_i$ is periodically soft updated by the most recent μ_i of the critic network. And $\bar{\boldsymbol{\theta}} = \left\{\bar{\theta}_1, \bar{\theta}_2, ..., \bar{\theta}_N\right\}$ is the parameter set of the actor target network.

2.3.4. Multi-agent soft actor critic

In the UAV swarm air-to-air combat scene, for each UAV, its action space is continuous, and its observation is incomplete. Enlightened by SAC algorithm and CTDE mechanism, a maximum entropy-based MASAC approach for UAV swarm air-to-air combat tasks has been proposed in Ref. 46. MASAC is an off-policy CTDE-based deep RL method. In the centralized training process, the critic network takes in the global information of all UAVs' actions and observations as input, and its output is the evaluation value of the global situation. In the process of execution, the critic network is ignored, and only the actor network is responsible for the output strategy. These measures are helpful for reducing the non-stationarity of the UAV swarm combat environments, and can remarkably stabilize learning.

The objective of MASAC is to find a set of stochastic policy that can maximize the expected cumulative reward and the entropy, i.e.,

$$J(\pi_i) = \sum \mathbb{E}_{\boldsymbol{o}_i \sim \rho_\theta, \boldsymbol{a}_i \sim \pi_i}[R(\boldsymbol{o}_i, \boldsymbol{a}_i) + \alpha \mathscr{H}(\pi_i(\cdot|\boldsymbol{o}_i))] \tag{22}$$

$$\pi^* = \left\{\arg\max_{\pi_i'} J(\pi_i), i = 1, 2, \cdots, N\right\} \tag{23}$$

where the temperature parameter α controls the relative importance of the entropy against the reward, and Eq. (22) will recover the more conventional objective as $\alpha \to 0$. A centralized learning method called Centralized Soft-Q (CSQ) can be derived from the objective. CSQ iteratively updates its Q-value via the following formula.

$$Q_i^{\text{soft}}(\boldsymbol{o}, \boldsymbol{a}) \leftarrow r_i + \gamma \mathbb{E}_{\boldsymbol{o}' \sim \rho_\theta}\left[V_i^{\text{soft}}(\boldsymbol{o}')\right], \ \forall \boldsymbol{o}, \boldsymbol{a} \tag{24}$$

$$V_i^{\text{soft}}(\boldsymbol{o}) \leftarrow \alpha \log \int_A \exp\left(\frac{1}{\alpha} Q_i^{\text{soft}}\left(\boldsymbol{o}, \tilde{\boldsymbol{a}}\right)\right) \mathrm{d}\tilde{\boldsymbol{a}}, \ \forall o \tag{25}$$

Consequently, for each UAV, the optimal policy in Eq. (23) could be written as

$$\pi_i^*(\boldsymbol{o}_i, \boldsymbol{a}_i) = \exp\left\{\left[\frac{1}{\alpha}\left(Q_i^{\text{soft}^*}(\boldsymbol{o}, \boldsymbol{a}) - V_i^{\text{soft}^*}(\boldsymbol{o})\right]\right\}$$

By minimizing the Kullback-Leibler (KL) divergence between the policy distribution and the CSQ distribution, we can obtain the optimal policy

$$\pi_i^* = \arg\min_{\pi_i'} \mathbb{E}_{\boldsymbol{o}_i\sim D}\left[D_{\mathrm{KL}}\left(\pi_i'(\cdot|\boldsymbol{o}_i)\|\frac{\exp\left(\frac{1}{\alpha}Q_i^{\mathrm{soft}}(\boldsymbol{o},\cdot)\right)}{Z_i(\boldsymbol{o})}\right)\right]$$

where the partition function $Z_i(\boldsymbol{o}) = \int_A \exp\left(\alpha^{-1}Q_i^{\mathrm{soft}}\left(\boldsymbol{o},\widetilde{\boldsymbol{a}}\right)\right)\mathrm{d}\,\widetilde{\boldsymbol{a}} = \exp\left(\alpha^{-1}V_i^{\mathrm{soft}}(\boldsymbol{o})\right)$ It normalizes the distribution and does not contribute to the gradient of the new policy.

Here, the critic network is denoted as the Q-value network who evaluates the global state-action pair in the training process. To prevent overestimation caused by bootstrap, the Q-value network is designed as a pseudo twin network, where the structure of the two networks is the same but their parameters are not shared. The loss function of the Q-value network is defined as follows.

$$\begin{cases} L(\mu_{i,j}) = \mathbb{E}_{\boldsymbol{o},\boldsymbol{a},r_i,\boldsymbol{o}'}\left[\left(Q_{i,j}^{\mathrm{soft}}(\boldsymbol{o},\boldsymbol{a}_i,\boldsymbol{a}_{-i};\mu_{i,j}) - y\right)^2\right] \\ j = 1,2 \\ y = r_i + \gamma\min\limits_{j=1,2}\bar{Q}_{i,j}^{\mathrm{soft}}(\boldsymbol{o}',\boldsymbol{a}';\bar{\mu}_{i,j})|_{\boldsymbol{a}'=\pi(\boldsymbol{o}';\theta)} \end{cases} \tag{26}$$

where $\bar{Q}_{i,j}^{\mathrm{soft}}$ denotes the twin critic target network of $Q_{i,j}^{\mathrm{soft}}$, and its network parameter $\bar{\mu}_{i,j}$ is soft updated with the parameter $\mu_{i,j}$ of the critic network.

On the other, a tractable parameterized policy $\pi_i(\boldsymbol{a}_i|\boldsymbol{o}_i;\theta_i)$ is provided to each UAV, and the policy network is represented as the actor network. The objective function of the actor network can be defined as

$$J_{\theta_i}(\pi_i) = \mathbb{E}_{\boldsymbol{o}_i\sim D}\left[D_{\mathrm{KL}}\left(\pi_i(\cdot|\boldsymbol{o}_i;\theta_i)\|\frac{\exp\left(\frac{1}{\alpha}\min_{j\in\{1,2\}}Q_{i,j}^{\mathrm{soft}}\left(\boldsymbol{o},\cdot;\mu_{i,j}\right)\right)}{Z_i\left(\boldsymbol{o};\mu_{i,j}\right)}\right)\right] \tag{27}$$

where the partition function $Z_i(\boldsymbol{o};\mu_{i,j})$ is used for normalizing the CSQ distribution. It can be omitted as it is irrelevant to action $\boldsymbol{a}_i$. Furthermore, we utilize a neural network transformation to reparameterize the policy, which can be denoted as

$$\boldsymbol{a}_i = f_{\theta_i}(\varepsilon_i;\boldsymbol{o}_i)$$

where ε_i is an input noise vector with Gaussian distribution. Thus the objective in Eq. (27) can be rewritten as

$$J_{\theta_i}(\pi_i) = \mathbb{E}_{\boldsymbol{o}_i\sim D,\varepsilon_i\sim N}\left[\log\pi_i(f_{\theta_i}(\varepsilon_i;\boldsymbol{o}_i)|\boldsymbol{o}_i;\theta_i) - \min_{j\in\{1,2\}}Q_{i,j}^{\mathrm{soft}}(\boldsymbol{o},\boldsymbol{a}_i,\boldsymbol{a}_{-i};\mu_{i,j})|_{\boldsymbol{a}_i=f_{\theta_i}(\varepsilon_i;\boldsymbol{o}_i)}\right]$$

Then the gradient of the objective function with respect to the actor network can finally be derived as

$$\begin{aligned}\nabla_{\theta_i}J_{\theta_i}(\pi_i) = {} & \nabla_{\theta_i}\log\pi_i(\boldsymbol{a}_i|\boldsymbol{o}_i;\theta_i) + (\nabla_{a_i}\log\pi_i(\boldsymbol{a}_i|\boldsymbol{o}_i;\theta_i) \\ & -\nabla_{\boldsymbol{a}_i}\min_{j\in\{1,2\}}Q_{i,j}^{\mathrm{soft}}(\boldsymbol{o},\boldsymbol{a}_i,\boldsymbol{a}_{-i};\mu_{i,j})|_{\boldsymbol{a}_i=f_{\theta_i}(\varepsilon_i;\boldsymbol{o}_i)})\nabla_{\theta_i}f_{\theta_i}(\varepsilon_i;\boldsymbol{o}_i)\end{aligned}$$

2.4. Performance metrics

To evaluate the performance of the above MADRL methods with respect to UAV swarm air-to-air combat, three performance metrics: the mean accumulative reward η_{MAR}, the mean hit rate η_{MHR} and the mean win rate η_{MWR}, are used in this paper.

To estimate the learning effect of UAVs, we use the mean accumulative reward

$$\eta_{\mathrm{MAR}} = \frac{1}{|B|}\sum_{b=1}^{|B|}\sum_{i=1}^{N}R_i \tag{28}$$

where $|B|$ denotes the number of episodes,N is the total number of UAVs, and R_i represents the accumulative reward of UAV i.

To metric the quality of the MADRL methods in more detail, we adopt the mean hit rate

$$\eta_{MHR} = \frac{1}{|B|}\sum_{i=1}^{|B|}\frac{h_i}{N^-} \tag{29}$$

where $|B|$ is the number of episodes, h_i is the number of hitting in the i-th episode, and N^- denotes the total number of opponent UAVs.

In the UAV swarm non-cooperative game model, the team that wipes out all UAVs in the opponent team will win the game. To metric the final result of UAV swarm air-to-air combat, we apply the mean win rate

$$\begin{aligned}\eta_{\mathrm{MWR}} &= \tfrac{1}{|B|}\sum_{k=1}^{|B|}I(k), \\ I(k) &= \begin{cases}1, & \text{Win in the } k\text{-}th \text{ episode} \\ 0, & \text{Otherwise}\end{cases}\end{aligned} \tag{30}$$

where $|B|$ is the number of episodes. Here $I(\cdot)$ is the indicator function. For the k-th episode, if the UAV swarm has win this episode then $I(k) = 1$, otherwise $I(k) = 0$. Obviously, the larger the value of the mean win rate is, the better the performance of the MADRL method is.

3. Simulation results and discussion

MPE, developed by OpenAI, is a popular multi-agent RL environment with a continuous observation space. [30] In this paper, incorporating the features of UAV swarm air-to-air combat, we construct an UAV swarm confrontation environment with MPE. Following common practices, [49,56–57] we assume that both the red and blue UAVs are at the same altitude level, and limit the airspace range of the simulation environment to a horizontal plane, i.e., a two-dimensional motion space, where Cartesian coordinate system is used. Here, horizontal coordinate $x \in [-2,2]$, and longitudinal coordinate y goes from -2 to 2 too. We perform simulations with a red UAV team against a blue UAV team, where the red team consists of 4 red UAVs while the blue team includes 8 blue UAVs. For simplicity, all UAVs are homogeneous, and the initial position of each UAV is randomly generated within the entire battlefield. When all UAVs in one team are destroyed, the non-cooperative game will be terminated. Table 1 shows the physical parameters for each UAV, and the hyperparameters of the UAV swarm non-cooperative game environment are displayed in Table 2.

Fig. 2 shows an illustration of the air-to-air combat between 4 red UAVs and 8 blue UAVs (4v8), in which the air-to-air combat process is exhibited with 8 scenarios. In Scenario-1, 4 red UAVs and 8 blue UAVs are randomly generated in a two-dimensional battlefield composed by continu-

Table 1 List of UAV physical parameters and their values.

Parameter	Value
Maximum attacking angle (°)	90
Maximum threatening angle (°)	90
Maximum attacking radius (km)	0.3
Maximum threatening radius (km)	0.3
Interference radius (km)	0.7
Search radius	0.6
Maximum acceleration	3.0
Maximum speed	5.0
Maximum roll angle (°)	30
Maximum course angle (°)	180
The size of UAV	0.04
The mass of UAV	1.2

Table 2 List of hyperparameters and their values.

Parameter	Value
UAVs number	12
Episodes number	10,000
Maximum length of episodes	200
Learning rate of the actor	1.5×10^{-4}
Learning rate of the critic	3×10^{-4}
Discount factor	0.99
Batch size	256
Temperature parameter	0.2
Replay buffer size	1×10^{6}
Update frequency	10
Target smoothing coefficient	5×10^{-4}
Optimizer	Aadm
Number of network layer	4

ous spaces; from the 2-th to the 7-th scenario, each UAV executes its actions according to the employed MADRL method and learns its policy; in scenario-8, all blue UAVs have been wiped out, the red team wins the combat and the confrontation is finished. Next, we will compare the performance of MASAC, MADDPG, Dec-SAC and Dec-DDPG in the UAV swarm confrontation environment.

The mean accumulative reward η_{MAR} is a crucial metric with respect to the learning effect of UAVs. To investigate the performance of these MADRL methods for η_{MAR}, we plot the relationship between η_{MAR} and episode (Fig. 3). This indicates that for all methods the value of η_{MAR} converges finally as the increment of episode, representing the effective learning to all UAVs. In Fig. 3(a), we set that the blue team only employs MADDPG while the red team uses MASAC, MADDPG, Dec-SAC and Dec-DDPG respectively. This shows that η_{MAR} value under MASAC is higher than that of other MADRL methods, which suggests that the red team obtains the best return under MASAC. Besides, when the blue team is fixed with Dec-DDPG, the red team employing MASAC also gets the best return and the value of η_{MAR} under MASAC is higher than that of other methods by a wide margin (Fig. 3 (b)). Similarly, MASAC outperforms the other MADRL methods when the blue team is fixed with Dec-SAC (Fig. 3 (d)). Since the critic network under MASAC captures the extra information from other UAVs, each UAV using MASAC can adjust its policy according to the global estimate value of the pseudo twin network, and thus red UAVs can acquire larger η_{MAR} values than that of other MADRL methods. On the other hand, when the blue team only uses MASAC while the red team employs four MADRL methods respectively (Fig. 3 (c)). One can see that the value of η_{MAR} under MASAC is slightly higher than that of Dec-SAC when episode value larger than 3000, and the value of η_{MAR} under Dec-SAC will con-

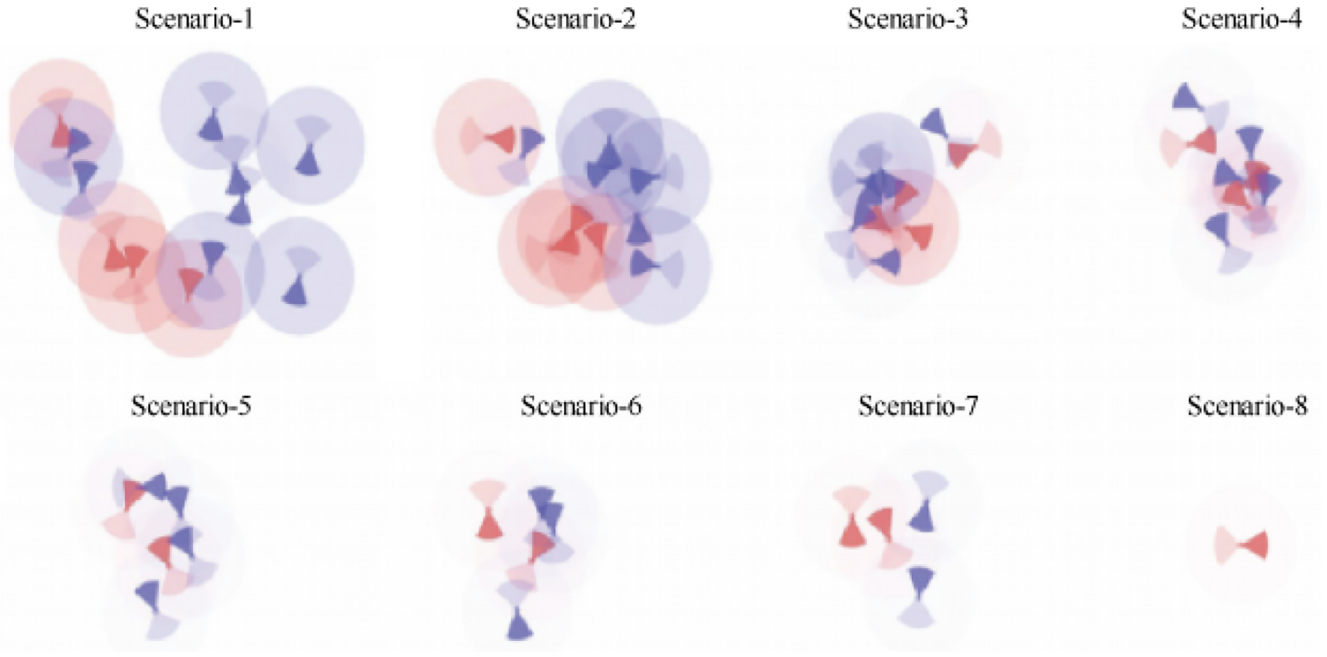


Fig. 2 An illustration of the air-to-air combat between 4 red UAVs and 8 blue UAVs (4v8).

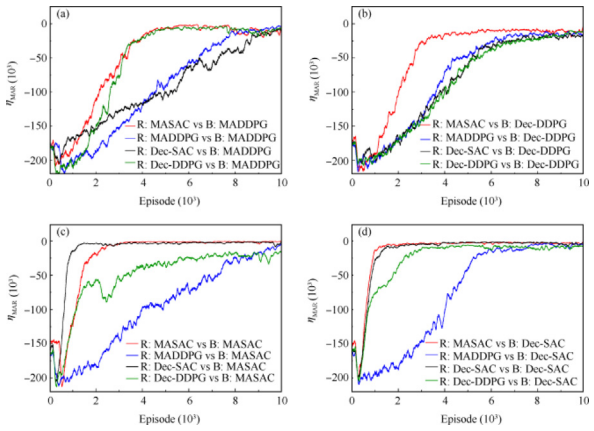


Fig. 3 Mean accumulative reward η_{MAR} as a function of episode, where the blue UAV team is orderly fixed with four different MADRL methods: (a) MADDPG; (b) Dec-DDPG; (c) MASAC and (d) Dec-SAC. Here R:MASAC vs. B:MADDPG means that red UAVs use MASAC while blue UAVs use MADDPG.

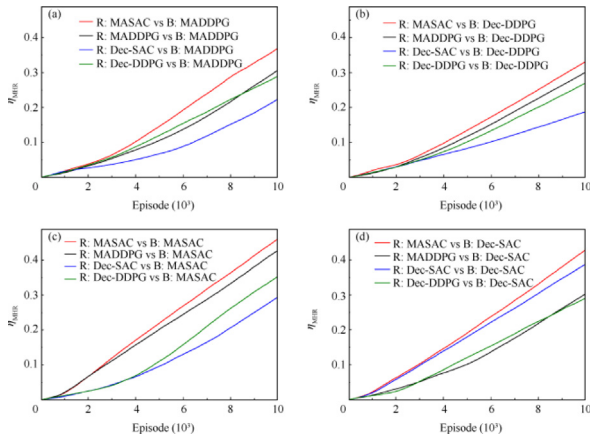


Fig. 4 Mean hit rate η_{MHR} of the red team as a function of episode, where the blue UAV team is orderly fixed with four different MADRL methods: (a) MADDPG; (b) Dec-DDPG; (c) MASAC and (d) Dec-SAC.

verge when episode value approaches to 1000, reflecting that the convergence speed of MASAC is relatively slower than that of Dec-SAC when the blue team also uses MASAC.

The mean hit rate η_{MHR} is an important metric that can evaluate the quality of employed MADRL methods in more detail. To investigate the efficiency of MADRL methods with respect to η_{MHR}, we plot the relationship between the mean hit rate η_{MHR} of the red team and episode, where the blue UAV team is orderly fixed with four different MADRL methods (Fig. 4). This shows that in all cases η_{MHR} value increases with the increment of episode, which implies that the attacking ability of red UAVs is gradually improved by learning. Especially, one can see that the value of η_{MHR} is always the largest under MASAC, meaning that UAVs' hit ability under MASAC is better than that of other MADRL methods.

The mean win rate η_{MWR} is a crucial metric who measures the final result of UAV swarm air-to-air combat. To study the effect of four MADRL methods with respect to the mean win rate, we display the relationship between the mean win rate η_{MWR} of the red team and episode, where the blue UAV team is still orderly fixed with four different MADRL methods (Fig. 5). This implies that η_{MWR} value increases with the increment of episode, illustrating that UAVs can learn effective policy. In Fig. 5(a), (b) and (c), the value of η_{MWR} under MASAC is the largest when the value of episode is greater than 8000, reflecting that the performance of MASAC is better than that of other methods by a large margin, which is in good accordance with Fig. 4. However, when the blue team is fixed with Dec-SAC, the value of η_{MWR} under MASAC is smaller than that of Dec-SAC (Fig. 5(d)), indicating that MASAC is no longer the best choice to win the combat when blue UAVs adopt Dec-SAC.

To further demonstrate the effect of MASAC, we list the highest win rate of red and blue UAV swarms under different scenarios in Table 3 and Table 4 respectively. One can see that R: MASAC versus B: MASAC can obtain the largest value of the highest win rate for both red and blue swarms, i.e., when two swarms all adopt MASAC, the non-cooperative game will converge to a Nash equilibrium with higher payoff, which further indicates that UAV swarm adopting MASAC can learn more effective policies.

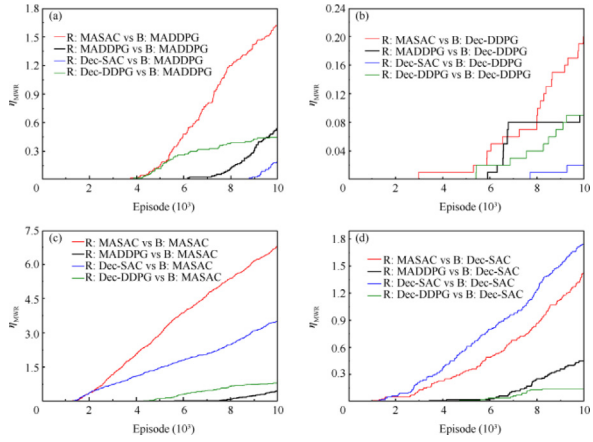


Fig. 5 Mean win rate η_{MWR} of the red team as a function of episode where the blue UAV team is orderly fixed with four different MADRL methods: (a) MADDPG; (b) Dec-DDPG; (c) MASAC and (d) Dec-SAC.

It is known that MASAC is a model-free, off-policy actor-critic RL method. Up to now, we have compared MASAC with other three model-free, off-policy actor-critic RL methods: Dec-DDPG, Dec-SAC and MADDPG. Note that MASAC is an off-policy RL method; COMA is a famous on-policy RL method; DQN is a classical value-based RL method. In order to further illustrate the advantage of MASAC in solving UAV swarm air-to-air combat game, the comparison between MASAC and traditional RL methods (COMA and DQN) via the mean win rate η_{MWR} of the blue swarm is plotted in Fig. 6. This shows that R: MASAC versus B: COMA can quickly learn more effective strategies and obtain a higher win rate when episode value is large, indicating

Table 3 The highest win rate of the red UAV swarm under different scenarios.

Red UAV	Blue UAV			
	B:MASAC	B:Dec-SAC	B:MADDPG	B:Dec-DDPG
R:MASAC	**10.8**	3.2	4.3	0.8
R:Dec-SAC	6.1	3.7	1.6	0.1
R:MADDPG	2.1	1.6	2.6	0.8
R:Dec-DDPG	2.5	0.9	1.8	0.8

Table 4 The highest win rate of the blue UAV swarm under different scenarios.

Red UAV	Blue UAV			
	B:MASAC	B:Dec-SAC	B:MADDPG	B:Dec-DDPG
R:MASAC	**87.4**	66.1	67.5	49.3
R:Dec-SAC	67.3	39.4	32.3	8.3
R:MADDPG	51.2	57.9	47.0	48.4
R:Dec-DDPG	39.3	13.7	21.3	23.9

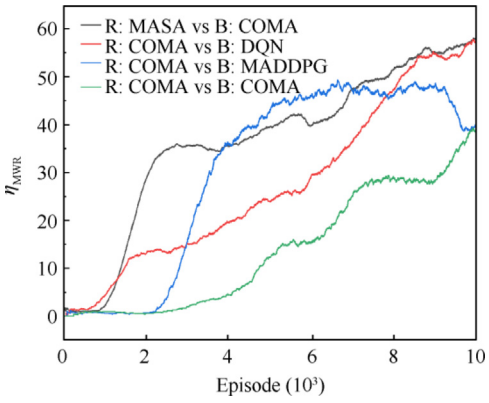


Fig. 6 Comparison between MASAC and traditional methods via the mean win rate η_{MWR} of the blue swarm.

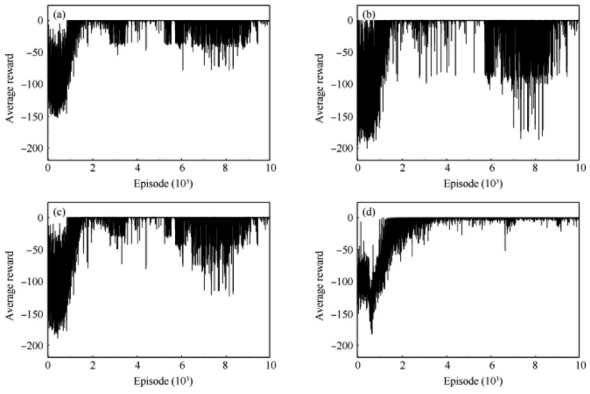


Fig. 7 Average reward of all UAVs with respect to different swarm sizes: (a) 3v3; (b) 4v4; (c) 5v5 and (d) 4v8.

that MASAC owns better performance than traditional methods in solving UAV swarm air-to-air combat game.

To investigate the generalization effect of MASAC, we plot the average reward of all UAVs under different swarm sizes (Fig. 7). This shows that in all cases the value of the average reward will converge when episode approaches 10000, illustrating that MASAC is of a well generalization effect with respect to different swarm sizes.

To further verify the generalization effect of MASAC, we perform simulations under different UAV physical parameters. Here the maximum speed of UAVs is changed to 3.0, and the interference radius is changed to 0.8. Fig. 8 plots the relationship between the mean accumulative reward η_{MAR} and episode. One can see that η_{MAR} curves for all UAVS are converged when episode value is large, which implies that UAVs can be well trained under manifold parameters, and MASAC is of the good generalization performance with respect to UAV physical parameters.

Finally, to ensure the practicability of MASAC and further verify whether the Nash equilibrium of the UAV swarm non-cooperative game under incomplete information has been reached? we investigate the convergence of MASAC by comparing the loss value of critic networks with respect to different UAVs. For MASAC, the critic network is denoted as the Q-value network which is designed as a pseudo twin network, and its loss value can be computed by Eq. (26), including Q1 loss and Q2 loss. The Q1 and Q2 losses are plotted in Fig. 9 and Fig. 10 respectively, which indicate that the loss value will first rise and then fall. Since the trajectory data in the experience replay buffer is not enough at the beginning, leading to the non-monotonic phenomena of the loss value. As the experience replay buffer has stored enough data the loss value will converge, i.e., the Nash equilibrium will be reached. Comparing Fig. 9(a) with Fig. 10(a), the tendency of the Q1 and Q2 losses for each red UAV is coincident, so do 8 blue UAVs

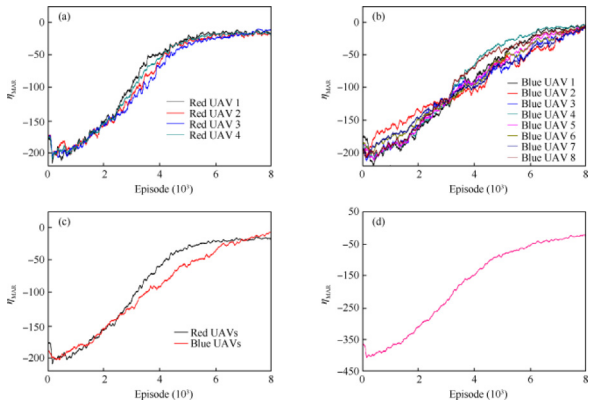


Fig. 8 Mean accumulative reward η_{MAR} as a function of episode under MASAC for different UAVs: (a) 4 red UAVs; (b) 8 blue UAVs; (c) red and blue UAVs and (d) all UAVs.

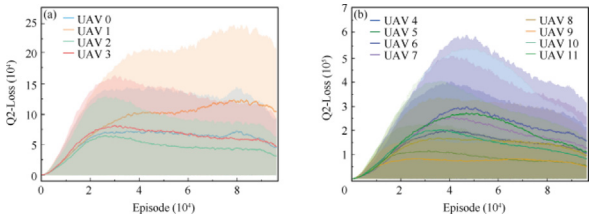


Fig. 9 Q1 loss as a function of episode under MASAC for different UAVs: (a) 4 red UAVs and (b) 8 blue UAVs.

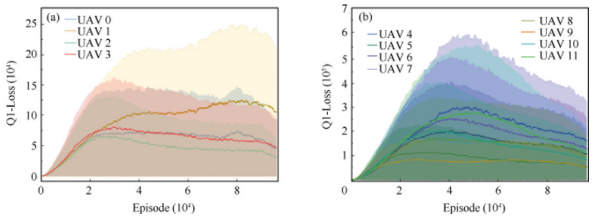


Fig. 10 Q2 loss as a function of episode under MASAC for different UAVs: (a) 4 red UAVs and (b) 8 blue UAVs.

(Fig. 9(b) and Fig. 10(b)). Especially, as the training progresses, the loss with respect to every UAV is converged. This means that two completely symmetric but parameter unshared pseudo twin critic networks can effectively reduce overestimation of Q values, guaranteeing that MASAC is practicable and the Nash equilibrium of the UAV swarm non-cooperative game can be reached.

4. Conclusions

To summarize, based on the flight dynamics of UAVs and the partially observable Markov game, we have proposed a MADRL-based UAV swarm non-cooperative game model under incomplete information, in which we construct the state and action spaces according to the features of UAV swarm air-to-air combat. Taking into account the incomplete continuous observation of individual UAV, we use MPE to build an UAV swarm combat environment with continuous observation space. The effects of four popular MADRL methods: Dec-DDPG, MADDPG, Dec-SAC and MASAC, are extensively investigated, the experimental results indicate that the performance of MASAC is better than that of other MADRL methods by a large margin. In general, UAVs employing MASAC can learn more effective policies, and obtain much higher hit rate and win rate. The generalization effect of MASAC are also be studied under different swarm sizes and physical parameters. The result indicates that MASAC possesses a high generalization ability with respect to manifold scenarios. Moreover, the practicability and convergence of MASAC are explored by investigating the loss value of Q-value networks, the results imply that MASAC is practicable and the Nash equilibrium of the UAV swarm non-cooperative game under incomplete information can be reached.

There are many different types of UAVs in real UAV swarm battles and the battlefield is actually a three-Dimensional (3D) airspace, in future work, by employing multi-agent reinforcement learning, we will incorporate the impact of UAV's flight altitude, and expand the simulation environment from 2D to 3D airspace with respect to large-scale, heterogeneous UAV swarm confrontation in more complex combat scenarios.

CRediT authorship contribution statement

Ershen WANG: Writing – original draft, Supervision, Resources, Methodology, Formal analysis, Conceptualization. **Fan LIU:** Writing – review & editing, Visualization, Software, Data curation. **Chen HONG:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Jing GUO:** Writing – original draft, Visualization, Software, Data curation. **Lin ZHAO:** Validation, Investigation, Formal analysis. **Jian XUE:** Supervision, Investigation, Funding acquisition, Conceptualization. **Ning HE:** Supervision, Project administration, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The work was supported by the National Key R&D Program of China (No. 2018AAA0100804), the National Natural Science Foundation of China (No. 62173237), the Academic Research Projects of Beijing Union University , China (Nos. SK160202103, ZK50201911, ZK30202107, ZK30202108), the SongShan Laboratory Foundation, China (No. YYJC062022017), the Applied Basic Research Programs of Liaoning Province, China (Nos. 2022020502-JH2/1013,

2022JH2/101300150), the Special Funds program of Civil Aircraft, China (No. 01020220627066) and the Special Funds program of Shenyang Science and Technology, China (No.22-322-3-34).

References

1. Lan YS, Zhang TT, Song AG. Adaptive structure modeling and prediction for swarm unmanned system. *Sci Sin-Inf* 2020;**50**(3):347–62.
2. Sun CY, Mu CX. Important scientific problems of multi-agent deep reinforcement learning. *Automatica Sinica* 2020;**46**(7):1301–12 [Chinese].
3. Wang ES, Guo J, Hong C, et al. Cooperative confrontation model of UAV swarm with random spatial networks. *J Beijing Univ Aeronaut Astronaut* 2023;**49**(1):10–6 [Chinese].
4. Wang ES, Guo J, Hong C, et al. UAV swarm air-ground engagement model with improved payoff. *J Nanjing Univ Aeronaut Astronaut* 2021;**53**(6):888–97 [Chinese].
5. Fan DD, Theodorou EA, Reeder J. Model-based stochastic search for large scale optimization of multi-agent UAV swarms. *2018 IEEE symposium series on computational intelligence (SSCI)*. Piscataway: IEEE Press; 2018.p.2216–22.
6. Zhou K, Wei RX, Zhang QR, et al. Learning system for air combat decision inspired by cognitive mechanisms of the brain. *IEEE Access* 1809;**8**:8129–44.
7. Song XY, Yang RP, Yin CS, et al. A cooperative aerial interception model based on multi-agent system for UAVs. *2021 IEEE 5th advanced information technology, electronic and automation control conference (IAEAC)*. Piscataway: IEEE Press; 2021. p. 873–82.
8. Zheng XM, Ma CY. An intelligent target detection method of UAV swarms based on improved KM algorithm. *Chin J Aeronaut* 2021;**34**(2):539–53.
9. Wang C, Wu LZ, Yan C, et al. Coactive design of explainable agent-based task planning and deep reinforcement learning for human-UAVs teamwork. *Chin J Aeronaut* 2020;**33**(11):2930–45.
10. Burch N, Johanson M, Bowling M. Solving imperfect information games using decomposition. *Proceedings of the 28th AAAI conference on artificial intelligence*. New York: ACM; 2014. p. 602–608.
11. Rowland M, Omidshafiei S, Tuyls K, et al. Multiagent evaluation under incomplete information. arXiv preprint:1909.09849, 2019.
12. Duan HB, Li P, Yu YX. A predator-prey particle swarm optimization approach to multiple UCAV air combat modeled by dynamic game theory. *IEEE/CAA J Autom Sin* 2015;**2**(1):11–8.
13. Huang CQ, Dong KS, Huang HQ, et al. Autonomous air combat maneuver decision using Bayesian inference and moving horizon optimization. *J Syst Eng Electron* 2018;**29**(1):86–97.
14. Li SY, Chen M, Wang YH, et al. Human-computer gaming decision-making method in air combat under an incomplete strategy set. *Sci Sin-Inf* 2022;**52**(12):2239.
15. Degrave J, Felici F, Buchli J, et al. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature* 2022;**602**(7897):414–9.
16. Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. *Nature* 2015;**518**(7540):529–33.
17. Wang ZY, Schaul T, Hessel M, et al. Dueling network architectures for deep reinforcement learning. *Proceedings of the 33rd international conference on international conference on machine learning* - Volume 48. New York: ACM; 2016. p. 1995–2003.
18. van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double Q-Learning. *Proceedings of the 13th AAAI conference on artificial intelligence*. New York: ACM; 2016. p. 2094–2100.
19. Huang Y, Wei GL, Wang YX. V-D D3QN: the variant of double deep Q-learning network with dueling architecture. *2018 37th Chinese control conference (CCC)*. Piscataway: IEEE; 2018. p. 9130–5.
20. Kapturowski S, Ostrovski G, Quan J, et al. Recurrent experience replay in distributed reinforcement learning. *Proceedings of the iternational conference on learn-ing representations*. New York: ACM; 2018.
21. Badia AP, Sprechmann P, Vitvitskyi A, et al. Never give up: Learning directed exploration strategies. arXiv preprint: 2003.06038, 2020.
22. Badia AP, Piot B, Kapturowski S, et al. Agent57: Outperforming the Atari human benchmark. In: *Proceedings of the 37th international conference on machine learning*. New York: ACM; 2020. p. 507–517.
23. Co-Reyes JD, Miao YJ, Peng DY, et al. Evolving reinforcement learning algorithms. arXivpreprint:2101.03958, 2021.
24. Williams RJ. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach Learn* 1992;**8**(3):229–56.
25. Sutton RS, McAllester D, Singh S, et al. Policy gradient methods for reinforcement learning with function approximation. *Proceedings of the 12th international conference on neural information processing systems*. New York: ACM;1999. p. 1057–63.
26. Mnih V, Badia AP, Mirza M, et al. Asynchronous methods for deep reinforcement learning. *Proceedings of the 33rd international conference on mchine learning* - Volume 48. 2016, New York: ACM; 2016. p. 1928–37.
27. Silver D, Lever G, Heess N, et al. Deterministic policy gradient algorithms. *Proceedings of the 31st international conference on international conference on machine learning* - Volume 32. New York: ACM; 2014:I–387–I–395.
28. Lillicrap TP, Hunt JJ, Pritzel A, et al. Continuous control with deep reinforcement learning. arXiv preprint: 1509.02971, 2015.
29. Haarnoja T, Zhou A, Abbeel P, et al. Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. arXiv preprint:1801.01290,2018.
30. Lowe R, Wu Y, Tamar A, et al. Multi-agent actor-critic for mixed cooperative-competitive environments. *Proceedings of the 31st international conference on neural information processing systems*. New York: ACM; 2017. p. 6382–93.
31. Yu C, Velu A, Vinitsky E, et al. The surprising effectiveness of PPO in cooperative, multi-agent games. arXiv preprint:2103.01955,2021.
32. Foerster J, Farquhar G, Afouras T, et al. Counterfactual multi-agent policy gradients. arXiv preprint:1705.08926,2018.
33. Guo T, Jiang N, Li BY, et al. UAV navigation in high dynamic environments: A deep reinforcement learning approach. *Chin J Aeronaut* 2021;**34**(2):479–89.
34. Liu P, Ma YF. A deep reinforcement learning based intelligent decision method for UCAV air combat. *Asian simulation conference*. Singapore: Springer; 2017. p. 274–286.
35. Toghiani-Rizi B, Kamrani F, Luotsinen LJ, et al. Evaluating deep reinforcement learning for computer generated forces in ground combat simulation. *2017 IEEE international conference on systems, man, and cybernetics (SMC)*. Piscataway: IEEE; 2017. p. 3433–8.
36. Yang QM, Zhu Y, Zhang JD, et al. UAV air combat autonomous maneuver decision based on DDPG algorithm. *2019 IEEE 15th international conference on control and automation (ICCA)*. Piscataway: IEEE; 2019. p. 37–42.
37. Kong WR, Zhou DY, Yang Z, et al. UAV autonomous aerial combat maneuver strategy generation with observation error based on state-adversarial deep deterministic policy gradient and inverse reinforcement learning. *Electronics* 2020;**9**(7):1121.

38. Fujimoto S, van Hoof H, Meger D. Addressing function approximation error in actor-critic methods. arXiv preprint:1802.09477, 2018.
39. Qiu XY, Yao ZY, Tan FW, et al. One-to-one air-combat maneuver strategy based on improved TD3 algorithm. *2020 Chinese automation congress (CAC)*. Piscataway: IEEE; 2020. p. 5719–25.
40. Cheng Y, Song Y. Autonomous dcision-mking generation of UAV based on soft actor-critic algorithm. *2020 39th Chinese control conference (CCC)*. Piscataway: IEEE; 2020. p. 7350–5.
41. Wei XL, Yang LF, Cao G, et al. Recurrent MADDPG for object detection and assignment in combat tasks. *IEEE Access* 2020;**8**:163334–43.
42. Kong WR, Zhou DY, Yang Z, et al. Maneuver strategy generation of UCAV for within visual range air combat based on multi-agent reinforcement learning and target position prediction. *Appl Sci* 2020;**10**(15):5198.
43. Kong WR, Zhou DY, Zhang K, et al. Air combat autonomous maneuver decision for one-on-one within visual range engagement base on robust multi-agent reinforcement learning. *2020 IEEE 16th international conference on control & automation (ICCA)*. Piscataway: IEEE; 2020. p. 506–12.
44. Kong WR, Zhou DY, Yang Z. Air combat strategies generation of CGF based on MADDPG and reward shaping. *2020 international conference on computer vision, image and deep learning (CVIDL)*. Piscataway: IEEE; 2020. p. 651–5.
45. Xiang L, Xie T. Research on UAV swarm confrontation task based on MADDPG algorithm. *2020 5th international conference on mechanical, control and computer engineering (ICMCCE)*. Piscataway: IEEE; 2020. p. 1513–8.
46. Wang ES, Liu F, Hong C, et al. MASAC-based confrontation game method of UAV clusters. *Sci Sin-Inf* 2022;**52**(12):2254.
47. Bai X, Lu CX, Bao QH, et al. An improved PPO for multiple unmanned aerial vehicles. *J Phys: Conf Ser* 2021;**1757**(1):012156.
48. Shi W, Feng YH, Cheng GQ, et al. Research on multi-aircraft cooperative air combat method based on deep reinforcement learning. *Acta Autom Sin* 2021;**47**(7):1610–23 [Chinese].
49. Zhang GY, Li Y, Xu XH, et al. Multiagent reinforcement learning for swarm confrontation environments. *International conference on intelligent robotics and applications*. Cham: Springer; 2019. p. 533–43.
50. Huang LW, Fu MS, Qu H, et al. A deep reinforcement learning-based method applied for solving multi-agent defense and attack problems. *Expert Syst Appl* 2021;**176**:114896.
51. Källström J, Heintz F. Agent coordination in air combat simulation using multi-agent deep reinforcement learning. *2020 IEEE international conference on systems, man, and cybernetics (SMC)*. New York: ACM; 2020. p. 2157–64.
52. Gong ZH, Xu Y, Luo DL. UAV cooperative air combat maneuvering confrontation based on multi-agent reinforcement learning. *Unmanned Syst* 2023;**11**(3):273–86.
53. Cao Y, Kou YX, Li ZW, et al. Autonomous maneuver decision of UCAV air combat based on double deep Q network algorithm and stochastic game theory. *Int J Aerosp Eng* 2023;**2023**:3657814.
54. Zhang JD, Wang DH, Yang QM, et al. Multi-dimensional air combat decision-making of UAV based on HRL. *ACTA ARMAMENTARII* 2022;**44**(6):1547–63 [Chinese].
55. Qiu Y, Zhao BQ, Zou J, et al. An autonomous guidance method of UAV in close air combat based on PPO algorithm. *Electron Opt Control* 2023;**30**(1):8–14 [Chinese].
56. McGrew JS, How JP, Williams B, et al. Air-combat strategy using approximate dynamic programming. *J Guid Contr Dyn* 2010;**33**(5):1641–54.
57. Ma W, Li H, Wang Z, et al. Close air combat maneuver decision based on deep stochastic game. *Systems Eng Electron* 2021;**43**(2):443–51 [Chinese].
58. Shapley LS. Stochastic games. *Proc Natl Acad Sci USA*, 1953;**39**(10):1095–100.
59. Littman ML. Markov games as a framework for multi-agent reinforcement learning. *Machine learning proceedings 1994*. Amsterdam: Elsevier; 1994. p. 157–63.
60. Lanctot M, Waugh K, Zinkevich M, et al. Monte Carlo sampling for regret minimization in extensive games. *Proceedings of the 22nd international conference on neural information processing systems*. New York: ACM; 2009. p. 1078–1086.
61. Heinrich J, Silver D. Deep reinforcement learning from self-play in imperfect-information games. arXiv preprint:1603.01121, 2016.
62. Nash J. Non-cooperative games. *Ann Math* 1951;**54**(2):286.
63. Yang YD, Wang J. An overview of multi-agent reinforcement learning from game theoretical perspective. arXiv preprint:2011.00583, 2020.
64. Bowling M, Veloso M. Rational and convergent learning in stochastic games. *Proceedings of the 17th international joint conference on atificial intelligence* - Volume 2. New York: ACM; 2001. p.1021–1026.
65. Bowling M, Veloso M. Multiagent learning using a variable learning rate. *Artif Intell* 2002;**136**(2):215–50.
66. Shoham Y, Powers R, Grenager T. If multi-agent learning is the answer, what is the question?. *Artif Intell* 2007;**171**(7):365–77.
67. Hao JY, Shao K, Li K, et al. Research and applications of game intelligence. *Sci Sin-Inf* 2023;**53**(10):1892.
68. Du Y, Li FX, Zandi HL, et al. Approximating Nash equilibrium in day-ahead electricity market bidding with multi-agent deep reinforcement learning. *J Mod Power Syst Clean Energy* 2021;**9**(3):534–44.
69. Albrecht SV, Christianos F, et al. *Multi-agent reinforcement learning: Foundations and modern approaches*. Pasadena:MIT Press; 2023.
70. Ghavamzadeh M, Mannor S, Pineau J, et al. Bayesian reinforcement learning: A survey. *Found Trends® Mach Learn* 2015;**8**(5–6):359–483.